Arbitrary Precision Calculator

Final Project Report for CS1023

Software Development Fundamentals

Submitted by:

Sarvan Naidu

CS24BTECH11024

Date: April 30, 2025

1 Abstract

This report discusses the design and implementation of an arbitrary-precision arithmetic library in Java, focusing on the key classes: `Ainteger`, `Afloat`, and `MyInfArith`. The library supports arithmetic operations (addition, subtraction, multiplication, division) on both integer and floating-point numbers with arbitrary precision. The design leverages string manipulation for arbitrary-precision handling and offers a detailed explanation of each file and their associated algorithms.

2 Introduction

In this project, we implement arbitrary-precision arithmetic for integers and floating-point numbers. This allows operations on numbers of arbitrary size or precision without being restricted by the limitations of standard Java primitive data types like `int` or `double`. The implementation consists of the following key components:

- `Ainteger` Class: Handles arbitrary-precision integer arithmetic.
- `Afloat` Class: Handles arbitrary-precision floating-point arithmetic.
- `MyInfArith` Class: Acts as the driver class to accept user inputs and perform the required operations.
- Ant Build Script: Facilitates building, compiling, and running the project.

2.1 Standard Types vs. Custom Arbitrary-Precision Types

Java's standard numeric types have fixed ranges that can be limiting for certain applications. Heres a comparison of their minimum and maximum values:

- `int`: A signed 32-bit integer with a range from -2^{31} to $2^{31} - 1$.
- `long`: A signed 64-bit integer with a range from -2^{63} to $2^{63} - 1$.
- `float`: A 32-bit floating-point number, which can represent values from approximately $\pm 3.4 \times 10^{38}$, with a precision of about 7 decimal digits.
- `double`: A 64-bit floating-point number, which can represent values from approximately $\pm 1.8 \times 10^{308}$, with a precision of about 15-16 decimal digits.

While these types cover a wide range of values, they are still limited by their size and precision.

In contrast, custom arbitrary-precision types, such as `Ainteger` and `Afloat`, can handle integer and floating-point numbers without predefined limits on range or precision. These types offer much greater flexibility for applications requiring high precision or large-scale calculations, effectively overcoming the limitations of Java's standard types.

3 File Descriptions

3.1 Ainteger.java

The `Ainteger` class provides arbitrary-precision integer arithmetic. It supports operations like addition, subtraction, multiplication, and division. The class works by processing the input digit-wise, using string manipulation to handle large numbers that exceed the limits of Java's native integer types.

- The class accepts integers as string inputs, allowing it to handle numbers of arbitrary size.
- Operations like addition, subtraction, multiplication, and division are performed digit by digit.
- The class includes mechanisms to validate input strings and remove leading or trailing zeroes.

3.2 Afloat.java

The `Afloat` class handles arbitrary-precision floating-point arithmetic. It stores the number as a string with separate integer and decimal parts, and uses the `Ainteger` class for handling the integer portions of the floating-point operations. The class provides similar arithmetic operations as `Ainteger`, with added complexity to manage decimal precision.

- The class initializes a floating-point number from a string, splitting it into integer and decimal parts.
- The class supports operations like addition, subtraction, multiplication, and division for floating-point numbers, handling both integer and decimal parts.
- The class includes methods for formatting the result and validating input strings, ensuring the precision of the floating-point calculations.

3.3 MyInfArith.java

The `MyInfArith` class serves as the entry point for running the arbitrary-precision arithmetic operations. It takes command-line arguments to determine the type of operation (addition, subtraction, multiplication, or division) and the operands (either integers or floating-point numbers). Based on the type, the corresponding class (`Ainteger` or `Afloat`) is used to perform the arithmetic operations.

- **Input Handling:** Accepts user input in the form of command-line arguments, which specify the type (integer or float), operation (add, sub, mul, div), and the two operands.
- **Operation Dispatch:** Depending on the operation requested, the appropriate method from `Ainteger` or `Afloat` is invoked.

3.4 Runner.py

- A Python script to compile and run `MyInfArith.java` with `aarithmetic.jar` in classpath.
- Accepts 4 command-line arguments: type, operation, operand1, and operand2.
- Uses `subprocess.run()` to handle Java compilation and execution.
- Displays result or error based on Java program output.

3.5 aarithmetic.jar

The `aarithmetic.jar` file is an essential part of the project, and it serves the following purposes:

- It includes the compiled `Ainteger.class` and `Afloat.class` files.
- It contains functionality for arbitrary-precision integer and floating-point arithmetic, supporting operations such as addition, subtraction, multiplication, and division on large numbers.
- The JAR file is created through an automated build process defined in the `build.xml` Ant file. This file compiles the Java classes and packages them into the JAR.
- It is intended for use in applications requiring high-precision numerical operations that cannot be handled by Java's standard integer and floating-point types.

3.6 Ant Build Script (build.xml)

The Ant build script automates the compilation and execution of the project. It defines the following targets:

- `clean`: Deletes all compiled class files and the JAR file.
- `compile`: Compiles all Java source files.
- `jar`: Packages the compiled classes into a JAR file for easier distribution and execution.
- `run`: Executes the `MyInfArith` class, passing the appropriate arguments to perform the arithmetic operations.

4 Algorithms

4.1 Ainteger Class

4.1.1 Addition

1. A check is made to determine the signs of the operands. If one number is negative, the operation is transformed into subtraction (i.e., adding the absolute value of the negative number to the other).
2. Aligning the numbers by adding zeroes to the shorter number.
3. Adding the numbers digit by digit, carrying over any excess value (similar to traditional addition).
4. Formatting the result and removing leading zeroes.

4.1.2 Subtraction

1. A sign check determines if one number is negative. If so, the operation is converted into addition by using the absolute value of the negative number.
2. Align the numbers by adding zeroes to the shorter number.
3. Subtract the numbers digit by digit, borrowing as needed.
4. Format the result by removing leading zeroes.

4.1.3 Multiplication

1. A sign check is done to see if one or both operands are negative. If one operand is negative, the result is negative; if both are negative, the result is positive.
2. The negative signs are removed and added to the result at last based on the sign check.
3. Treating the numbers as large integers, multiplying them digit by digit.
4. Carrying over any excess value during multiplication.
5. Adjusting the result to account for the number of digits in the operands.
6. Formatting the result and removing leading zeroes.

4.1.4 Division

1. A sign check is done first: if exactly one operand is negative, the final quotient is marked negative.
2. Both numbers are converted to their absolute values for the division process.
3. Digits from the dividend are added one by one to form a temporary value.
4. At each step, the largest multiple of the divisor that fits into the temporary value is determined and added to the quotient.
5. This process continues until all digits are processed or the remaining part becomes smaller than the divisor.
6. The remainder is stored separately, and leading zeroes are removed from the final quotient.

4.2 Afloat Class

4.2.1 Addition

The addition of two arbitrary-precision floating-point numbers involves:

1. If one number is negative, treat the operation as subtraction.
2. Align the numbers by counting the digits after the decimal in both numbers and pad the shorter number with zeroes to match the longer one.
3. Remove the decimal points from both numbers, treating the integer and decimal parts as one large integer (e.g., 9.99 becomes 999 and 8.75 becomes 875).

4. Add the numbers using integer addition (using the `Ainteger` class).
5. Reinsert the decimal point into the result after addition, based on the original decimal places of the two numbers.

4.2.2 Subtraction

Subtraction of two arbitrary-precision floating-point numbers follows these steps:

1. If one number is negative, treat the operation as addition.
2. Align the numbers by counting and padding the decimal places to match the longer one.
3. Remove the decimal points and perform subtraction on the resulting integers.
4. Reinsert the decimal point into the result after subtraction.

4.2.3 Multiplication

Multiplication of two arbitrary-precision floating-point numbers involves:

1. Check for negative signs, and treat the result sign accordingly.
2. Count the decimal places in both numbers and add them together.
3. Remove the decimal points, treating both numbers as large integers.
4. Multiply the integers using the integer multiplication method (using the `Ainteger` class).
5. Reinsert the decimal point at the position determined by the total number of decimal places.

4.2.4 Division

Division of two arbitrary-precision floating-point numbers follows these steps:

1. Check for negative signs, and treat the result sign accordingly.
2. Remove the decimal points from both numbers and treat them as large integers.

3. Use long division to compute the quotient, adding digits to the quotient until it reaches a maximum length or the division is complete.
4. Reinsert the decimal point in the quotient at the appropriate position.
5. To maintain precision, the result is rounded to 30 significant digits. If the result extends beyond this, any additional digits are truncated, and trailing zeros after the decimal point are removed.

5 Testing

Correctness of the functions of these classes was mainly tested manually, with comprehensive test cases for both integer and float operations.

5.1 Ainteger

1. $1234 + 5678 = 6912$
2. $78 + (-56) = 22$
3. $5678 - 1234 = 4444$
4. $98879 - (-97767) = 196646$
5. $5344 \times 98008 = 523754752$
6. $-878 \times 99 = -86922$
7. $-878 \times 999876565 = -877891624070$
8. $100 \div 789 = 0$
9. $997783738 \div (-976) = -1022319$
10. $997783738 \div$ hi = Exception (Invalid input)
11. 99778373 ÷ -99-7 = Exception (Invalid input)

5.2 Afloat

1. $12.34 + 56.78 = 69.12$
2. $78 + (-56.5) = 21.5$
3. $56.78 - 12.34 = 44.44$

4. $98.879 - (-97.767) = 196.646$
5. $5.344 \times 9.8008 = 52.3754752$
6. $-87.8 \times 9.9 = -869.22$
7. $2 \div 8.00 = 0.25$
8. $-76.27221 \div (-8.87762) = 8.591515518798957378216233630184$
9. $32.5 \div 0.0 =$ Exception (Division by zero)
10. $25qbc + 8.2 =$ Exception (Invalid input)

6 Build and Run

6.1 Building the package (.JAR):

- `ant clean` followed by `ant`

6.2 Executing via MyInfArith:

- `javac MyInfArith.java`
- `java MyInfArith <int/float> <add/sub/mul/div> <operand1> <operand2>`

6.3 Executing via runner.py:

- `python runner.py <int/float> <add/sub/mul/div> <operand1> <operand2>`

6.4 Executing via aarithmetic.jar:

- `javac -cp .:<add the absolute path to the .jar package here> <relative path to your desired file>`
- `java -cp .:<absolute path to .jar package> <relative path to your file>`